

PERTEMUAN KE-7: METODE K-MEDOID

A. TUJUAN

Setelah menempuh praktikum ini diharapkan mahasiswa mampu

1. menjelaskan cara kerja dari metode K-Medoid.
2. membuat program metode K-Medoid dengan menggunakan Bahasa Pemrograman Python.
3. menggunakan metode K-Medoid untuk analisis *cluster*.

B. MATERI

K-Medoids atau Partitioning Around Method (PAM) adalah metode cluster non hirarki yang merupakan varian dari metode K-Means. K-Medoids hadir untuk mengatasi kelemahan K-Means yang sensitif terhadap outlier karena suatu objek dengan suatu nilai yang besar mungkin secara substansial menyimpang dari distribusi data (Jiawei & Kamber, 2006). Hal ini didasarkan pada penggunaan medoids bukan dari pengamatan mean yang dimiliki oleh setiap cluster, dengan tujuan mengurangi sensitivitas dari partisi sehubungan dengan nilai ekstrim yang ada dalam dataset (Vercellis, 2009)

K-Medoids menggunakan metode pengelompokan partisi untuk mengelompokkan sekumpulan n objek menjadi sejumlah k cluster. Algoritma ini menggunakan objek pada kumpulan objek yang mewakili sebuah cluster. Objek yang mewakili sebuah cluster disebut dengan medoids. Medoid merupakan objek yang letaknya terpusat di dalam suatu cluster sehingga robust terhadap outlier. Cluster dibangun dengan menghitung kedekatan yang dimiliki antara medoids dengan objek non medoids. Sama halnya dengan K-Means, metode K-Medoid lebih menguntungkan diterapkan pada data yang sangat besar.

Algoritma K-Medoids

1. Inisialisasi

1. Tentukan jumlah kluster, k .
2. Pilih k medoid secara acak dari dataset sebagai pusat awal kluster.

2. Penugasan Data ke Kluster

1. Hitung jarak antara setiap titik data dengan masing-masing medoid menggunakan metrik jarak, seperti **Euclidean Distance** atau **Manhattan Distance**.
2. Tetapkan setiap titik data ke kluster yang memiliki medoid terdekat.

3. Optimasi Medoid

1. Untuk setiap kluster, pilih titik dalam kluster sebagai kandidat medoid baru.
2. Hitung **Total Biaya Perubahan Medoid (Cost)** dengan mengganti medoid lama dengan kandidat medoid baru: $Cost = \sum_{i=1}^N d(x_i, M')$ di mana:
 - o x_i adalah titik dalam kluster,
 - o M' adalah kandidat medoid baru,
 - o $d(x_i, M')$ adalah jarak antara titik x_i dengan kandidat medoid baru.
3. Jika penggantian medoid menurunkan total biaya (total jarak dalam kluster menjadi lebih kecil), maka ubah medoid ke kandidat tersebut.

4. Iterasi Sampai Konvergensi

1. Ulangi langkah 2 dan 3 sampai tidak ada perubahan medoid atau hingga jumlah iterasi maksimum tercapai.
2. Jika tidak ada perubahan medoid dalam iterasi berturut-turut, maka algoritma dianggap konvergen.

Contoh 7.1

Diketahui dataset seperti pada Tabel 7.1

Tabel 7.1. Dataset 1

ID	x_{i1}	x_{i2}
x_1	2	6
x_2	3	4
x_3	3	8

x_4	5	8
x_5	7	5
x_6	6	4

Kelompokkan data tersebut menjadi 2 kluster ($k = 2$) dengan metode k-Medoid.

Jawaban

1. Inisialisasi Medoid Awal

Misalkan dipilih $M_1 = x_1 = (2,6)$ dan $M_2 = x_5 = (7,5)$ sebagai medoid awal.

2. Hitung Jarak Setiap Titik ke Medoid

Gunakan **Euclidean Distance** untuk menghitung jarak antar titik:

$$d(M_1, x_2) = \sqrt{(2-3)^2 + (6-4)^2} = \sqrt{1+4} = \sqrt{5} \approx 2.24$$

$$d(M_1, x_3) = \sqrt{(2-3)^2 + (6-8)^2} = \sqrt{1+4} = \sqrt{5} \approx 2.24$$

$$d(M_1, x_4) = \sqrt{(2-5)^2 + (6-8)^2} = \sqrt{9+4} = \sqrt{13} \approx 3.61$$

$$d(M_1, x_5) = \sqrt{(2-7)^2 + (6-5)^2} = \sqrt{25+1} = \sqrt{26} \approx 5.10$$

$$d(M_1, x_6) = \sqrt{(2-6)^2 + (6-4)^2} = \sqrt{16+4} = \sqrt{20} \approx 4.47$$

$$d(M_2, x_1) = \sqrt{(7-2)^2 + (5-6)^2} = \sqrt{25+1} = \sqrt{26} \approx 5.10$$

$$d(M_2, x_2) = \sqrt{(7-3)^2 + (5-4)^2} = \sqrt{16+1} = \sqrt{17} \approx 4.12$$

$$d(M_2, x_3) = \sqrt{(7-3)^2 + (5-8)^2} = \sqrt{16+9} = \sqrt{25} = 5$$

$$d(M_2, x_4) = \sqrt{(7-5)^2 + (5-8)^2} = \sqrt{4+9} = \sqrt{13} \approx 3.61$$

$$d(M_2, x_6) = \sqrt{(7-6)^2 + (5-4)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$$

3. Penugasan Titik ke Kluster Terdekat

Berdasarkan perhitungan jarak, setiap titik dikategorikan ke dalam kluster terdekat:

- **Kluster 1 (Medoid: M_1)** $\rightarrow \{x_1, x_2, x_3, x_4\}$
- **Kluster 2 (Medoid: M_2)** $\rightarrow \{x_5, x_6\}$

4. Optimasi Medoid

Coba ganti medoid dalam kluster dengan titik lain dalam kluster dan hitung total jarak dalam kluster untuk melihat apakah ada pengurangan biaya.

Misalkan dalam Kluster 1 kita coba mengganti medoid dari $x_1 \rightarrow x_3$ dan menghitung ulang total biaya.

Jika penggantian medoid mengurangi total biaya, maka kita mengganti medoid, jika tidak, kita pertahankan medoid awal.

6. Konvergensi

- Jika tidak ada perubahan medoid dalam iterasi berikutnya, maka algoritma berhenti.
- Jika ada perubahan, ulangi langkah (3) hingga (5) sampai tidak ada perubahan medoid lagi.

Hasil Akhir

Setelah beberapa iterasi, algoritma menghasilkan kluster berikut:

- **Kluster 1 (Medoid: x_3)** $\rightarrow \{A, B, C, D\}$
- **Kluster 2 (Medoid: x_6)** $\rightarrow \{E, F\}$

Contoh 7.2

Buatlah program K-medoid dengan Python tanpa pustaka.

Jawaban

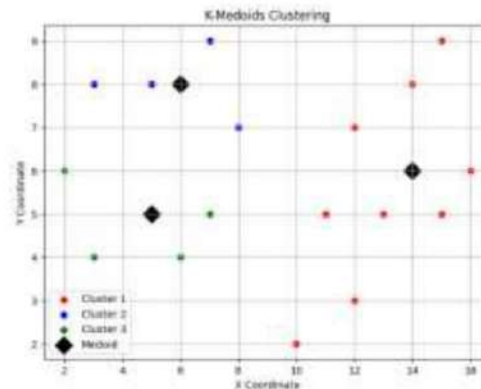
Kode Program 7.1. Program metode k-medoid

No	Kode Program
1	<code>import numpy as np</code>
2	<code>import matplotlib.pyplot as plt</code>
3	<code>def euclidean_distance(p1, p2):</code>
4	<code> """Menghitung jarak Euclidean antara dua titik."""</code>
5	<code> return np.sqrt(np.sum((np.array(p1) - np.array(p2)) ** 2))</code>
6	<code>def total_cost(cluster, medoid):</code>
7	<code> """Menghitung total jarak dari semua titik dalam kluster ke medoid."""</code>
8	<code> return sum(euclidean_distance(point, medoid) for point in cluster)</code>
9	<code>def assign_clusters(data, medoids):</code>
10	<code> """Menetapkan setiap titik data ke medoid terdekat."""</code>
11	<code> clusters = {i: [] for i in range(len(medoids))}</code>


```

12     for point in data:
13         distances = [euclidean_distance(point, medoid) for medoid in medoids]
14         closest_medoid = np.argmin(distances)
15         clusters[closest_medoid].append(point)
16     return clusters
17 def k_medoids(data, k, max_iter=100):
18     """Implementasi K-Medoids dari nol."""
19     # Inisialisasi: Pilih K medoid secara acak
20     medoid_indices = np.random.choice(len(data), k, replace=False)
21     medoids = [data[i] for i in medoid_indices]
22     for _ in range(max_iter):
23         clusters = assign_clusters(data, medoids)
24         new_medoids = []
25         for cluster_idx in clusters:
26             cluster_points = clusters[cluster_idx]
27             best_medoid = medoids[cluster_idx]
28             best_cost = total_cost(cluster_points, best_medoid)
29
30             # Coba setiap titik dalam kluster sebagai kandidat medoid baru
31             for candidate in cluster_points:
32                 cost = total_cost(cluster_points, candidate)
33                 if cost < best_cost:
34                     best_cost = cost
35                     best_medoid = candidate
36             new_medoids.append(best_medoid)
37         # Jika medoid tidak berubah, hentikan iterasi
38         if np.array_equal(new_medoids, medoids):
39             break
40         medoids = new_medoids
41     return medoids, assign_clusters(data, medoids)
42 def plot_clusters(data, medoids, clusters):
43     """Visualisasi hasil clustering K-Medoids."""
44     colors = ['r', 'b', 'g', 'c', 'm', 'y', 'k'] # Warna kluster (maks 7)
45     plt.figure(figsize=(8, 6))
46     for cluster_idx, points in clusters.items():
47         points = np.array(points)
48         plt.scatter(points[:, 0], points[:, 1], color=colors[cluster_idx],
49 label=f'Cluster {cluster_idx+1}')
50         # Plot medoid dengan simbol berbeda
51         medoids = np.array(medoids)
52         plt.scatter(medoids[:, 0], medoids[:, 1], color='black', marker='D', s=150,
53 label='Medoid')
54     plt.xlabel('X Coordinate')
55     plt.ylabel('Y Coordinate')
56     plt.title('K-Medoids Clustering')
57     plt.legend()
58     plt.grid()
59     plt.show()
60 # Contoh Data (20 titik dengan 3 kluster)
61 data = np.array([
62     [2, 6], [3, 4], [3, 8], [5, 8], [7, 5], [6, 4], [8, 7], [5, 5], [6, 8], [7,
63     9],
64     [10, 2], [12, 3], [11, 5], [13, 5], [14, 6], [15, 5], [12, 7], [14, 8], [16,
65     6], [15, 9]
66 ])
67 # Jalankan K-Medoids dengan K = 3
68 k = 3
69 final_medoids, final_clusters = k_medoids(data, k)
70 # Tampilkan hasil klasterisasi
71 print("Medoid Akhir:", final_medoids)
72 for i, cluster in final_clusters.items():
73     print(f"Kluster {i+1}: {cluster}")
74 # Plot hasil klasterisasi
75 plot_clusters(data, final_medoids, final_clusters)

```



Gambar 7.1. Plot data setelah dikelompokkan

Contoh 7.3

Buatlah program K-medoid dengan Python dengan pustaka.

Jawaban

Kode Program 7.2. Program metode k-medoid dengan pustaka.

No	Kode Program
1	<code>import numpy as np</code>
2	<code>import matplotlib.pyplot as plt</code>
3	<code>from sklearn_extra.cluster import KMedoids</code>
4	<code># Data (20 titik dengan 3 kluster)</code>
5	<code>data = np.array([</code>
6	<code> [2, 6], [3, 4], [3, 8], [5, 8], [7, 5], [6, 4], [8, 7], [5, 5], [6, 8], [7,</code>
7	<code> 9],</code>
8	<code> [10, 2], [12, 3], [11, 5], [13, 5], [14, 6], [15, 5], [12, 7], [14, 8], [16,</code>
9	<code> 6], [15, 9]</code>
10	<code>])</code>
11	<code># Jalankan K-Medoids dengan K = 3</code>
12	<code>k = 3</code>
13	<code>kmedoids = KMedoids(n_clusters=k, random_state=42, method='pam') # Gunakan</code>
14	<code>metode PAM (Partitioning Around Medoids)</code>
15	<code>kmedoids.fit(data)</code>
16	<code># Ambil hasil clustering</code>
17	<code>medoids = kmedoids.cluster_centers_</code>
18	<code>labels = kmedoids.labels_</code>
19	<code># Visualisasi hasil clustering</code>
20	<code>def plot_clusters(data, labels, medoids):</code>
21	<code> """Visualisasi hasil clustering K-Medoids."""</code>
22	<code> colors = ['r', 'b', 'g', 'c', 'm', 'y', 'k'] # Warna kluster (maks 7)</code>
23	<code> plt.figure(figsize=(8, 6))</code>
24	<code> for cluster_idx in range(k):</code>
25	<code> cluster_points = data[labels == cluster_idx]</code>
26	<code> plt.scatter(cluster_points[:, 0], cluster_points[:, 1],</code>
27	<code> color=colors[cluster_idx], label=f'Cluster {cluster_idx+1}')</code>
28	<code></code>
29	<code> # Plot medoid dengan simbol berbeda</code>
30	<code> plt.scatter(medoids[:, 0], medoids[:, 1], color='black', marker='D', s=150,</code>
31	<code> label='Medoid')</code>
32	<code> plt.xlabel('X Coordinate')</code>
33	<code> plt.ylabel('Y Coordinate')</code>
34	<code> plt.title('K-Medoids Clustering with sklearn_extra')</code>
35	<code> plt.legend()</code>
36	<code> plt.grid()</code>
37	<code> plt.show()</code>
38	<code># Tampilkan hasil clustering</code>
39	<code>print("Medoid Akhir:\n", medoids)</code>
40	<code># Plot hasil klasterisasi</code>
41	<code>plot_clusters(data, labels, medoids)</code>

C. TUGAS/LATIHAN SOAL

1. Gunakan metode k-Medoid untuk pengelompokan data pelanggan dengan melakukan preprocessing terlebih dahulu.
2. Tentukan k yang terbaik dengan metode Elbow
3. Berikan analisis terhadap cluster yang sudah diperoleh.